

Implicit conversion traits and utility functions

- [Introduction](#)
- [Revision History](#)
- [Discussion](#)
- [Proposal Candidate](#)
- [Resolved Issues](#)
- [Proposed Resolution](#)
- [Bibliography](#)
- [Sample Implementation](#)

Introduction

This proposal became a rather minimalistic proposal by suggesting only a single new trait `is_nothrow_convertible`, based on the feedback the author got from both the LEWG and LWG discussions.

By means of these suggested new facilities, this paper attempts to resolve the existing library issue [LWG 2040](#) and provides a tool that would allow to resolve [LWG 2999](#).

Revision History

Changes since [P0758R0](#):

- Rebases to the reference working draft [N4750](#)
- Eliminates the suggestions to add `is_trivially_convertible`, `decay_copy`, and `implicit_cast`. Reasons for this decision are two-fold:
 1. [LEWG straw polls](#) indicated no convincing motivation to add either `decay_copy` or `implicit_cast`.
 2. During the Rapperswil 2108 LWG session concerns were expressed that adding `is_trivially_convertible` could easily be a cause of nasty compile-time errors or ODR-violations (Similar to those of the pre-existing `is_trivially_constructible` trait), when at least one of the template arguments would be a type (such as a pointer type) which is complete according to the core language, but whose referred to type is incomplete. Another concern was that [P0758R0](#) did not present a concrete use-case demonstrating the motivation for another intrinsic-depending trait. The author therefore decided to split-off `is_trivially_convertible` from this proposal, suggesting to defer a potential addition by an independent proposal.
- The proposal no longer recommends to specify an SG10 feature-testing macro name.

Discussion

This paper revision does not repeat the sections [Discussion](#), [2. is_trivially_convertible](#), [3. decay_copy](#), and [4. implicit_cast](#) of its predecessor [P0758R0](#).

Please refer to the previous paper regarding general background and (extended) rationale.

Proposal Candidate

`is_nothrow_convertible`

The `is_nothrow_convertible` trait

```
template <class From, class To>
struct is_nothrow_convertible;
```

is presumably one of the most often asked for traits related to `is_convertible`. One of the earliest official notes of the lack of that feature occurred during the publication of the [N3255](#) proposal that shortly before the C++11 finalization attempted to standardize a new Standard

Library function template `decay_copy` as replacement for the existing `DECAY_COPY` pseudo-function. Among other reasons (especially the lateness of the feature request), this proposal failed, because it couldn't provide the correct conditional exception specification, concluding:

"What we would need is `std::is_nothrow_convertible`."

Shortly after C++11 standardization, [LWG 2040](#) was filed requesting the addition of `is_nothrow_convertible` and `is_trivially_convertible`, but is since then in a kind of zombie state.

There are several concrete use-cases for the `is_nothrow_convertible` trait:

1. It could be used to specify the correct `noexcept` expression for a `decay_copy` replacement template for `DECAY_COPY`, which would resolve one important aspect of [LWG 2999](#).
2. It could be used to restore the valueable (now conditionally) `noexcept` specification for several non-throwing `basic_string` functions that had been "string_view"-ified, as described by [LWG 2946](#). As example consider adjusting the currently suggested wording changes for the following `find` signature:

```
template <class T>
size_type find(basic_string_view<charT, traits> sv const T& t, size_type pos = 0) const noexcept(see below);
```

-1- *Effects:* Creates a variable, `sv`, as if by `basic_string_view<charT, traits> sv = t;` and then determines the lowest position `xpos`, if possible, such that both of the following conditions hold: [...]

-2- *Returns:* `xpos` if the function can determine such a value for `xpos`. Otherwise, returns `npos`.

-?- *Remarks:* This function shall not participate in overload resolution unless `is_convertible v<const T&, basic_string_view<charT, traits>>` is true and `is_convertible v<const T&, const charT*>` is false. The expression inside `noexcept` is equivalent to:

```
is_nothrow_convertible v<const T&, basic_string_view<charT, traits>>
```

3. It could be used to define exception-correct — either by user-code or by the Standard Library — a fundamental `implicit_cast` utility function, as suggested by.

Resolved Issues

If the proposed resolution will be accepted, the following library issues will be resolved:

Number	Description
2040	Missing type traits related to <code>is_convertible</code>

Proposed resolution

The proposed wording changes refer to [N4750](#).

1. Change 23.15.2 [meta.type.synop], header `<type_traits>` synopsis, as indicated:

```
namespace std {
    [...]
    // 23.15.6 [meta.rel], type relations
    template <class T, class U> struct is_same;
    template <class Base, class Derived> struct is_base_of;
    template <class From, class To> struct is_convertible;
    template <class From, class To> struct is_nothrow_convertible;
    [...]

    // 23.15.6 [meta.rel], type relations
    template <class T, class U> inline constexpr bool is_same_v
        = is_same<T, U>::value;
    template <class Base, class Derived> inline constexpr bool is_base_of_v
        = is_base_of<Base, Derived>::value;
    template <class From, class To> inline constexpr bool is_convertible_v
        = is_convertible<From, To>::value;
    template <class From, class To> inline constexpr bool is_nothrow_convertible_v
    = is_nothrow_convertible<From, To>::value;
    [...]
}
```

2. Change 23.15.6 [meta.rel], Table 44 — "Type relationship predicates", as indicated:

Table 44 — Type relationship predicates

Template	Condition	Comments
...		
template <class From, class To> struct is_convertible;	see below	From and To shall be complete types, arrays of unknown bound, or cv void types.
template <class From, class To> struct is_nothrow_convertible;	is convertible v<From, To> is true and the conversion, as defined by is_convertible, is known not to throw any exceptions ([expr.unary.noexcept]).	From and To shall be complete types, arrays of unknown bound, or cv void types.
...		

Bibliography

[N3255 Lawrence Crowl, Daniel Krügler: "C++ Decay Copy"](#)

[P0705R0 Tony Van Eerd: "Implicit and Explicit Conversions"](#)

Sample Implementation

Example implementation for the is_nothrow_convertible type trait.

```
#include <type_traits> // std::enable_if, ...
#include <utility>      // std::forward, std::declval

namespace xstd {
namespace details {
template <class From, class To, bool =
    std::disjunction<
        std::is_void<From>, std::is_function<To>, std::is_array<To>
    >::value
>
struct do_is_nothrow_convertible
{
    using type = std::is_void<To>;
};

struct do_is_nothrow_convertible_impl
{
    template <class To>
    static void test_aux(To) noexcept;

    template <class From, class To>
    static std::bool_constant<noexcept(test_aux<To>(std::declval<From>()))>
    test(int);

    template <class, class>
    static std::false_type
    test(...);
};

template <class From, class To>
struct do_is_nothrow_convertible<From, To, false>
{
    using type = decltype(do_is_nothrow_convertible_impl::test<From, To>(0));
};

} // details

template <class From, class To>
struct is_nothrow_convertible :
    details::do_is_nothrow_convertible<From, To>::type
{ };

template <class From, class To>
inline constexpr bool is_nothrow_convertible_v
    = is_nothrow_convertible<From, To>::value;
```

```
} // xstd
```